TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM

KHOA TOÁN - TIN HỌC

BÁO CÁO THỰC HÀNH

Môn học: Xử lý dữ liệu lớn

Bài thực hành L03

Hive: truy vấn SQL trên HDFS, phân vùng, định dạng lưu trữ cột và bucketing

Sinh viên thực hiện: Nguyễn Thị Huyền Trang
Mã số sinh viên: 23280092
Lớp: 23KDL

TP. Hồ Chí Minh, Tháng 10 năm 2026

Mục lục

1 Mục tiêu và môi trường **2**

2 Thực hiện và kết quả **3**
2.1 Khởi động cụm và đưa CSV lên HDFS . . . 3
2.2 Bảng ngoài và nội dung Metastore . . . 4
2.3 Bảng phân vùng ORC và Parquet . . . 5
2.4 So sánh thời gian truy vấn CSV, ORC và Parquet . . . 9
2.5 Cắt tỉa phân vùng . . . 11
2.6 Bucketing . . . 12
2.7 Phần mở rộng: thống kê và bảng ACID . . . 13

3 Trả lời câu hỏi checkpoint **16**
3.1 Metastore giữ gì, HDFS giữ gì, và `DROP TABLE` xóa những gì? . . . 16
3.2 Vì sao `zones` cần `OpenCSVSerde` còn `trips_csv` chỉ cần `ROW FORMAT DELIMITED`? . . . 16
3.3 Vì sao ORC và Parquet nhỏ hơn CSV 4–5 lần? . . . 16
3.4 Vì sao Q1 chạy nhanh hơn trên ORC và Parquet? . . . 16
3.5 Phân vùng tĩnh khác phân vùng động ra sao, và vì sao mặc định là `strict`? . . . 17
3.6 Phân vùng theo `pulocationid` cộng ngày, hoặc theo `tpep_pickup_datetime`, thì sao? . . . 17
3.7 Khi nào dùng partitioning, khi nào dùng bucketing, và vì sao số tệp bucket cố định? . . . 17
3.8 Những dòng nào rơi vào phân vùng lạ, và pipeline thật nên xử lý ra sao? . . . 17

4 Kết luận **18**

Tài liệu tham khảo **18**

Phụ lục: Trích đoạn mã nguồn **19**

Danh mục hình

1 Khởi động cụm Hive (`docker compose ps`) và tải tệp CSV cùng bảng tra cứu zone lên HDFS. . . . 3
2 Giao diện web HiveServer2 (cổng 10002): danh sách truy vấn đã hoàn tất, tất cả chạy trên Tez. . . . 4
3 Kết quả truy vấn bảng ngoài và nội dung Metastore (PostgreSQL). . . . 5
4 `SHOW PARTITIONS` và số chuyến theo `pickup_month`. . . . 6
5 Thư mục warehouse `/user/hive/warehouse/taxi.db` trên giao diện NameNode. . . . 6

6 Năm thư mục phân vùng của bảng `trips_orc` trên giao diện NameNode. . . . 7
7 `DESCRIBE FORMATTED trips_orc` và 18 dòng nằm ngoài tháng 01/2024. . . . 8
8 Kích thước CSV, ORC và Parquet trên HDFS. . . . 9
9 Lần chạy thứ hai của `03_compare_formats.sql`. . . . 10
10 Kết quả `EXPLAIN DEPENDENCY` và `EXPLAIN EXTENDED`. . . . 12
11 Kết quả bucketing: mô tả bảng, lấy mẫu một bucket và vị trí các dòng JFK. . . . 13
12 Bốn tệp bucket của `trips_bucketed` trên giao diện NameNode. . . . 13
13 Phần mở rộng: thống kê cột và các thao tác ACID. . . . 15
14 Các thư mục `delta_*` và `delete_delta_*` của bảng `zone_notes` trên giao diện NameNode. . . . 15

Danh mục bảng

1 Môi trường thực nghiệm. . . . 2
2 Dữ liệu đầu vào. . . . 3
3 Nội dung bảng `TBLS` và `SDS` trong Metastore sau bước tạo bảng ngoài. . . . 5
4 Phân vùng của `trips_orc` sau khi nạp bằng phân vùng động. . . . 5
5 Kích thước lưu trữ cùng dữ liệu tháng 01/2024 theo định dạng. . . . 9
6 Thời gian truy vấn theo định dạng (số liệu dùng để so sánh là lần chạy thứ hai). 9
7 Kết quả Q1 (giống nhau trên CSV, ORC và Parquet). . . . 11
8 Kết quả Q2: số chuyến và doanh thu theo borough. . . . 11
9 Kết quả kiểm chứng cắt tỉa phân vùng. . . . 11
10 Các tệp bucket của `trips_bucketed`. . . . 12
11 Các giao dịch trên bảng ACID `zone_notes` và thư mục delta tương ứng. . . . 14
12 Các dòng có thời gian đón khách nằm ngoài tháng 01/2024. . . . 18
13 Danh mục tệp nộp kèm trong `submission/L03/`. . . . 19

1 Mục tiêu và môi trường

Bài thực hành tạo bảng ngoài Hive trên CSV ở HDFS, chuyển sang bảng phân vùng ORC và Parquet, so sánh kích thước và thời gian truy vấn, kiểm chứng cắt tỉa phân vùng bằng `EXPLAIN`, tạo bảng bucketing, và làm thêm hai phần mở rộng (thống kê cột, bảng ACID). Cả cụm chạy bằng Docker Compose trên một máy (Bảng 1).

Bảng 1. Môi trường thực nghiệm.

Thành phần	Phiên bản / cấu hình
Hive	4.2.1 (`apache/hive:4.2.1`), HiveServer2 + Metastore độc lập
Bộ máy thực thi	Tez, chế độ cục bộ bên trong HiveServer2
Cơ sở dữ liệu Metastore	PostgreSQL 18 (database `metastore`)

Thành phần	Phiên bản / cấu hình
HDFS	3.4.3, 1 NameNode + 1 DataNode (hệ số nhân bản 1)
Máy khách	Beeline qua `run_sql.sh` (in thời gian từng câu lệnh)

Đề bài gợi ý dùng mẫu 1 triệu dòng, nhưng chúng tôi dùng nguyên tháng 01/2024 của dữ liệu taxi vàng New York (Bảng 2), nên các con số trong bài lớn hơn số tham khảo của đề.

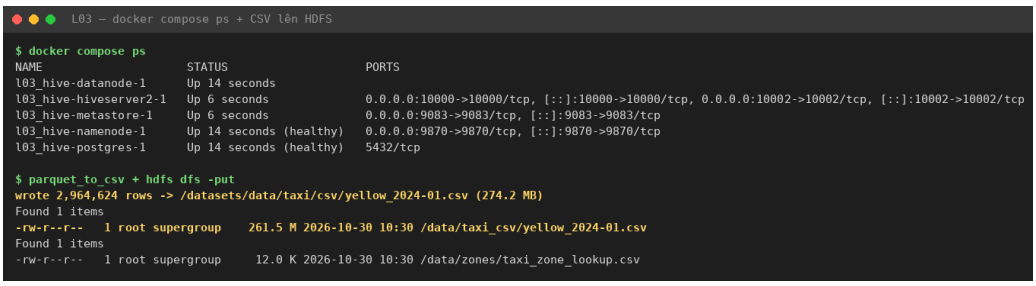
2 Thực hiện và kết quả

2.1 Khởi động cụm và đưa CSV lên HDFS

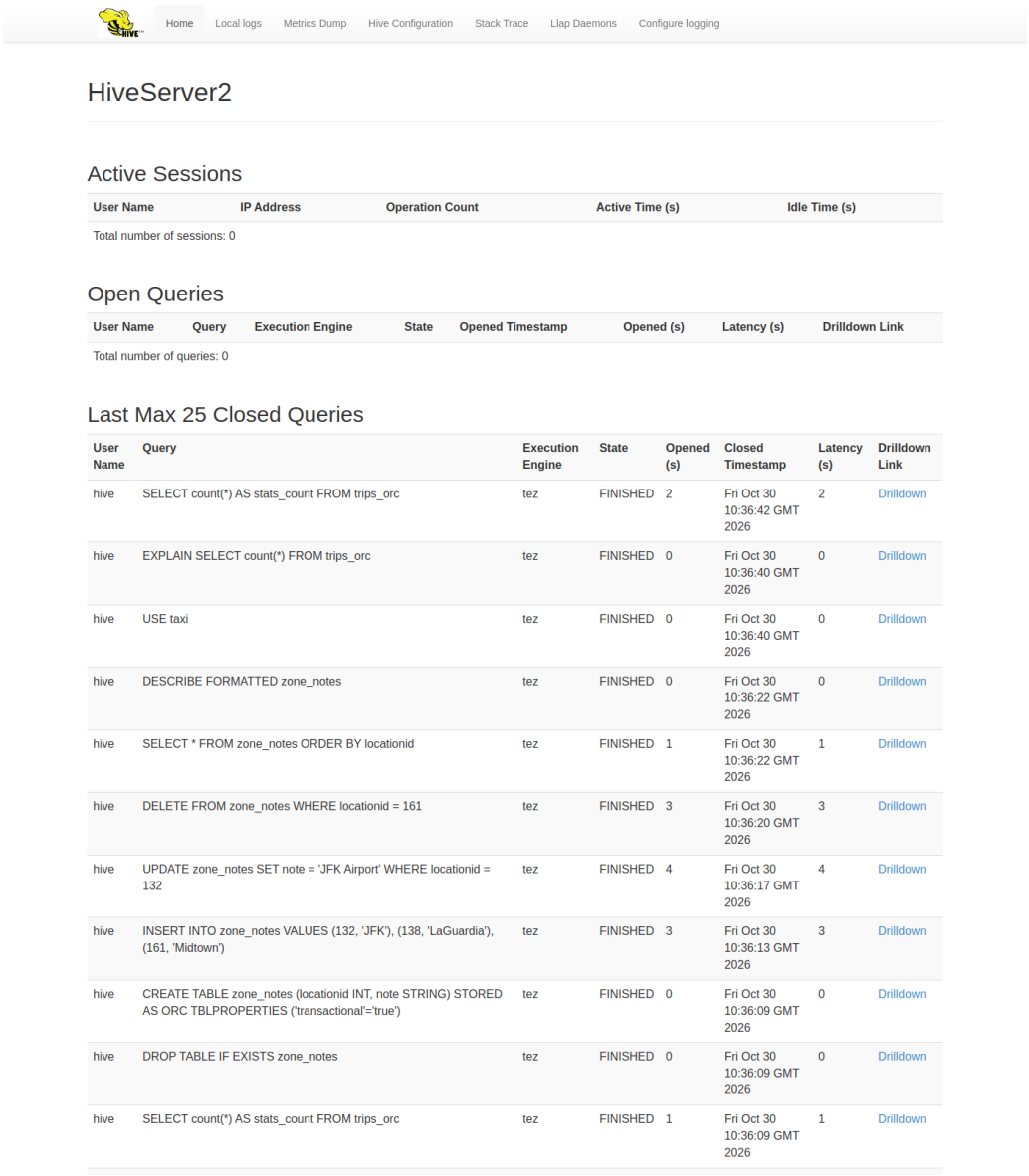
Năm container đều `Up`, NameNode và PostgreSQL ở trạng thái `healthy` (Hình 1). Tệp `yellow_tripdata_2024-01.parquet` được chuyển sang CSV không có dòng tiêu đề rồi tải lên `/data/taxi_csv`, cùng bảng tra cứu zone lên `/data/zones` (Bảng 2). Giao diện web HiveServer2 ở cổng 10002 liệt kê các truy vấn đã chạy xong, tất cả trên `tez` (Hình 2).

Bảng 2. Dữ liệu đầu vào.

Tệp trên HDFS	Kích thước	Số dòng
`/data/taxi_csv/yellow_2024-01.csv`	274,217,811 B (274.2 MB, 261.5 MiB)	2,964,624
`/data/zones/taxi_zone_lookup.csv`	12.0 K	265 zone + 1 dòng tiêu đề



Hình 1. Khởi động cụm Hive (`docker compose ps`) và tải tệp CSV cùng bảng tra cứu zone lên HDFS.



Hình 2. Giao diện web HiveServer2 (cổng 10002): danh sách truy vấn đã hoàn tất, tất cả chạy trên Tez.

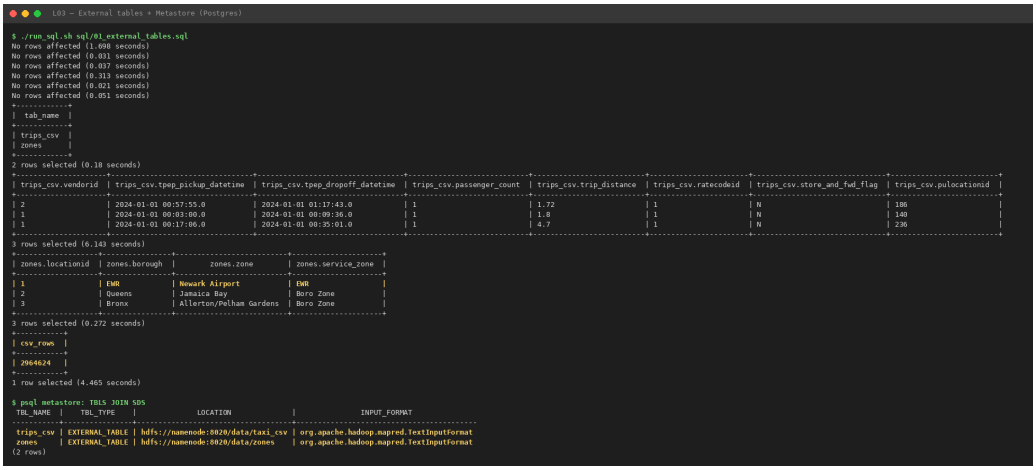
2.2 Bảng ngoài và nội dung Metastore

`01_external_tables.sql` tạo `trips_csv` với `ROW FORMAT DELIMITED` và `zones` với `OpenCSVSerde` cộng `skip.header.line.count=1` (mã ở Phụ lục). Cả hai đọc được ngay. `SELECT * FROM zones LIMIT 3` trả về `1 | EWR | Newark Airport | EWR`, `2 | Queens | Jamaica Bay | Boro Zone` và `3 | Bronx | Allerton/Pelham Gardens | Boro Zone`, tức dòng tiêu đề và dấu ngoặc kép đã được bỏ. `count(*)` trên `trips_csv` ra `csv_rows = 2,964,624` sau 4.465 s, khớp số dòng của tệp.

Truy vấn thẳng `TBLS JOIN SDS` trong PostgreSQL (Bảng 3, Hình 3) chỉ thấy tên bảng, loại bảng, vị trí và lớp định dạng đầu vào; không có dòng dữ liệu nào trong Metastore.

Bảng 3. Nội dung bảng `TBLS` và `SDS` trong Metastore sau bước tạo bảng ngoài.

TBL_NAME	TBL_TYPE	LOCATION	INPUT_FORMAT
trips_csv	EXTERNAL_TABLE	`hdfs://namenode:8020/data/taxi_csv`	`org.apache.hadoop.mapred.TextInputFormat`
zones	EXTERNAL_TABLE	`hdfs://namenode:8020/data/zones`	`org.apache.hadoop.mapred.TextInputFormat`



Hình 3. Kết quả truy vấn bảng ngoài và nội dung Metastore (PostgreSQL).

2.3 Bảng phân vùng ORC và Parquet

`02_partitioned_tables.sql` bật `hive.exec.dynamic.partition.mode=nonstrict`, tính `pickup_month` bằng `date_format(..., 'yyyy-MM')` ở cột cuối của `SELECT`, rồi nạp `trips_orc` (ZLIB) và `trips_parquet` (Snappy). `INSERT OVERWRITE` ghi 2,964,624 dòng vào `trips_orc` trong 42.761 s và vào `trips_parquet` trong 45.134 s. Chúng tôi chờ 4 phân vùng như kết quả mẫu của đề, nhưng `SHOW PARTITIONS` trả về 5 (Bảng 4, Hình 4): dữ liệu cả tháng có thêm phân vùng `2024-02`.

Bảng 4. Phân vùng của `trips_orc` sau khi nạp bằng phân vùng động.

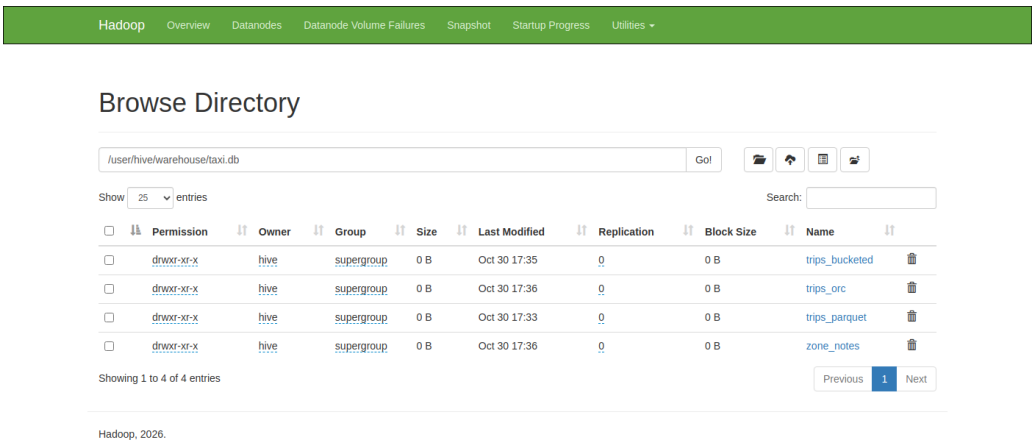
pickup_month	Số chuyến	Tệp ORC	Kích thước tệp
2002-12	2	`000007_0`	1,637 B
2009-01	3	`000004_0`	1,922 B
2023-12	10	`000001_0`	2,234 B
2024-01	2,964,606	`000005_0`	52,621,888 B
2024-02	3	`000005_0`	1,993 B

```
L03 — Dynamic partitioning ORC + Parquet

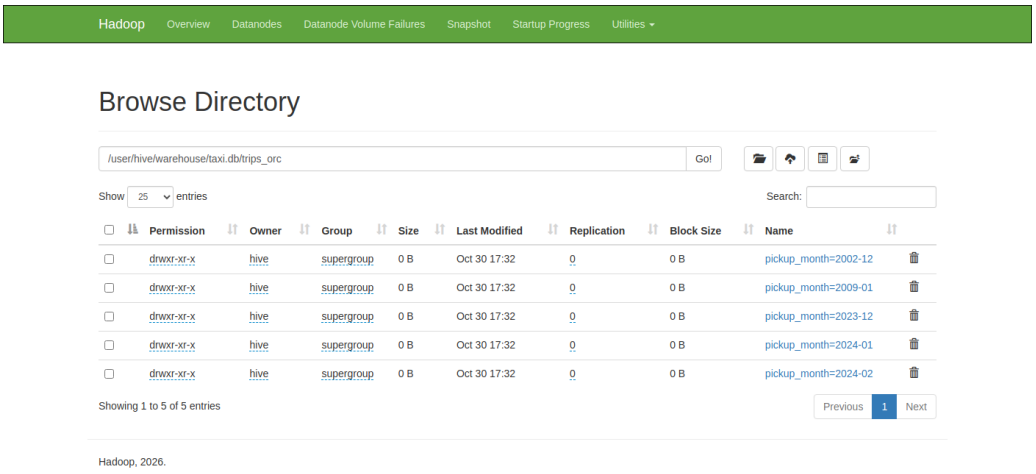
$ ./run_sql.sh sql/02_partitioned_tables.sql
No rows affected (0.06 seconds)
No rows affected (0.006 seconds)
No rows affected (0.004 seconds)
No rows affected (0.017 seconds)
No rows affected (0.07 seconds)
2,964,624 rows affected (42.761 seconds)
No rows affected (0.017 seconds)
No rows affected (0.052 seconds)
2,964,624 rows affected (45.134 seconds)
+-----------------------+
|       partition       |
+-----------------------+
| pickup_month=2002-12  |
| pickup_month=2009-01  |
| pickup_month=2023-12  |
| pickup_month=2024-01  |
| pickup_month=2024-02  |
+-----------------------+
5 rows selected (0.077 seconds)
+---------------+----------+
| pickup_month  |  trips   |
+---------------+----------+
| 2002-12       | 2        |
| 2009-01       | 3        |
| 2023-12       | 10       |
| 2024-01       | 2964606  |
| 2024-02       | 3        |
+---------------+----------+
5 rows selected (2.24 seconds)
```

Hình 4. `SHOW PARTITIONS` và số chuyến theo `pickup_month`.

Trên giao diện NameNode, thư mục `taxi.db` có bốn bảng `trips_bucketed`, `trips_orc`, `trips_parquet`, `zone_notes` (Hình 5), và `trips_orc` có năm thư mục con `pickup_month=...` (Hình 6). Bốn phân vùng nhỏ chỉ có một tệp khoảng 2 KB.

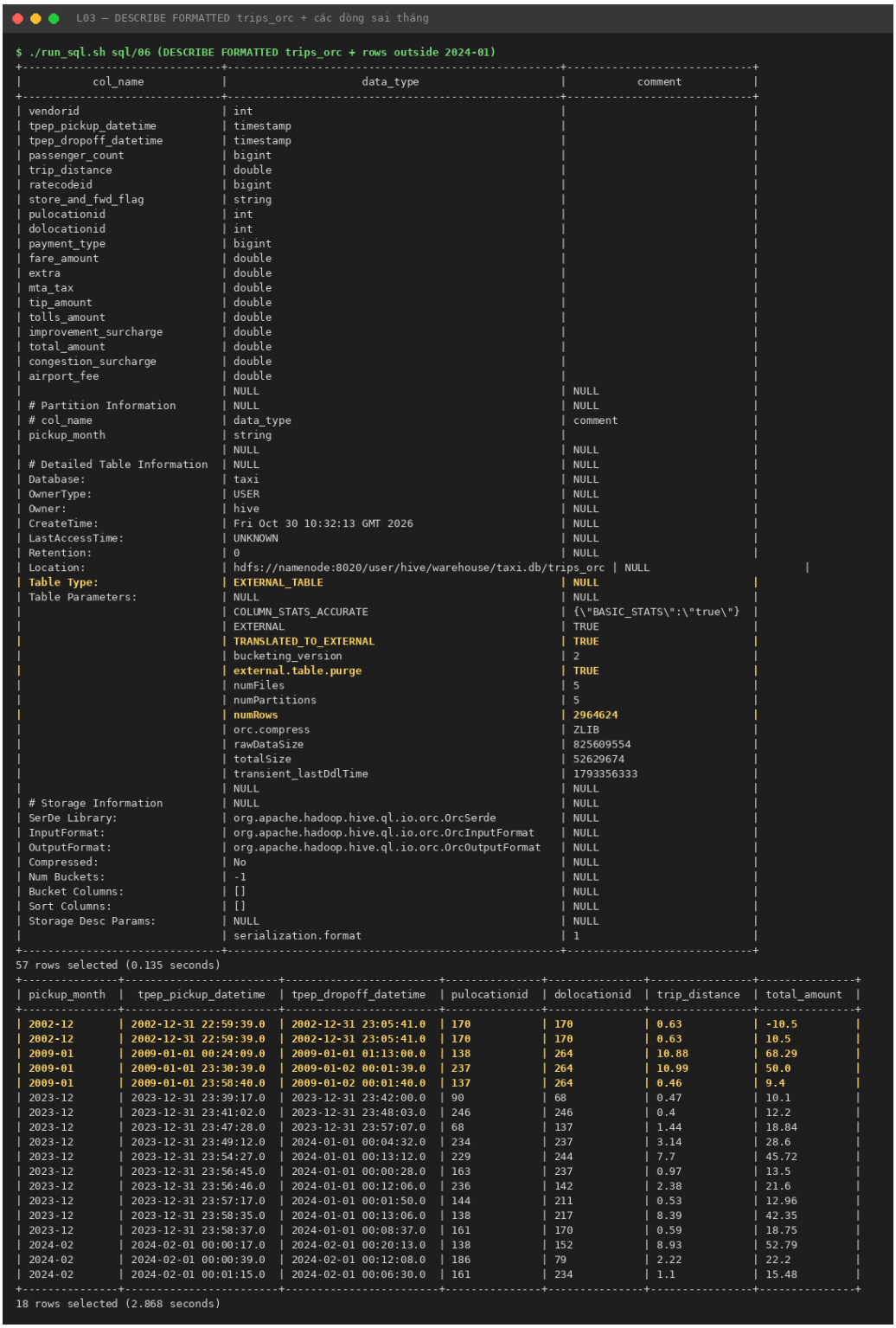


Hình 5. Thư mục warehouse `/user/hive/warehouse/taxi.db` trên giao diện NameNode.



Hình 6. Năm thư mục phân vùng của bảng `trips_orc` trên giao diện NameNode.

`DESCRIBE FORMATTED trips_orc` (Hình 7) ghi `Table Type: EXTERNAL_TABLE` cùng `TRANSLATED_TO_EXTERNAL=TRUE` và `external.table.purge=TRUE`. Lý do là Hive 4 chỉ chấp nhận bảng quản lý dạng ACID, nên một câu `CREATE TABLE ... STORED AS ORC` thường bị Metastore chuyển thành bảng ngoài nằm trong warehouse [1]. Thống kê trong Metastore: `numFiles=5`, `numPartitions=5`, `numRows=2964624`, `rawDataSize=825609554`, `totalSize=52629674`. Hình này cũng liệt kê 18 dòng có `pickup_month` khác `2024-01`, phân tích ở Mục 3.8.

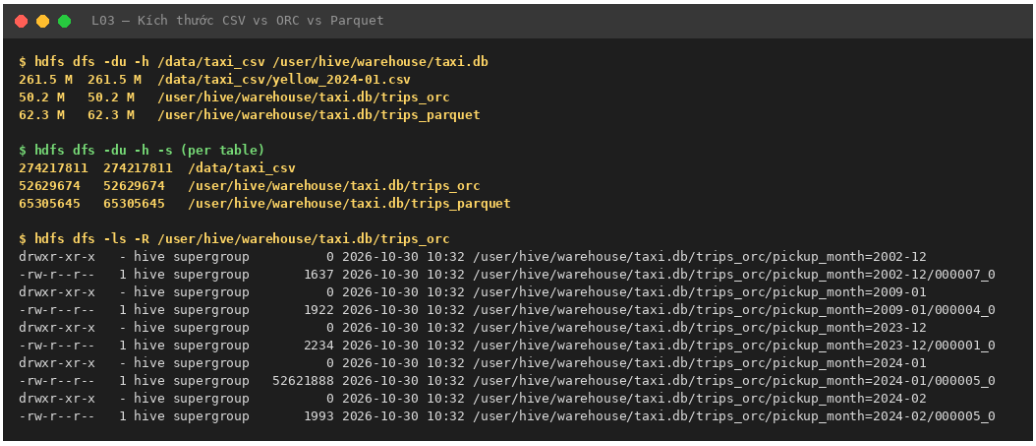


Hình 7. `DESCRIBE FORMATTED trips_orc` và 18 dòng nằm ngoài tháng 01/2024.

Kích thước đo bằng `hdfs dfs -du -s` nằm ở Bảng 5 và Hình 8. Tỉ lệ nén tính bằng $r = S_{\text{CSV}}/S_{\text{định dạng}}$, ví dụ $274{,}217{,}811/52{,}629{,}674 \approx 5.2$.

Bảng 5. Kích thước lưu trữ cùng dữ liệu tháng 01/2024 theo định dạng.

Định dạng	Kích thước (byte)	Kích thước (MiB)	Nhỏ hơn CSV
CSV (text)	274,217,811	261.5	1×
ORC + ZLIB	52,629,674	50.2	**5.2×**
Parquet + Snappy	65,305,645	62.3	4.2×
ORC bucketed (chỉ 5 cột)	25,271,072	24.1	10.9×



Hình 8. Kích thước CSV, ORC và Parquet trên HDFS.

ORC + ZLIB nhỏ hơn Parquet + Snappy khoảng 19 %, hợp lý vì ZLIB thiên về tỉ lệ nén còn Snappy thiên về tốc độ. Bảng bucketed chỉ giữ 5 trên 19 cột nên con số 10.9× không đem so với ba dòng trên được.

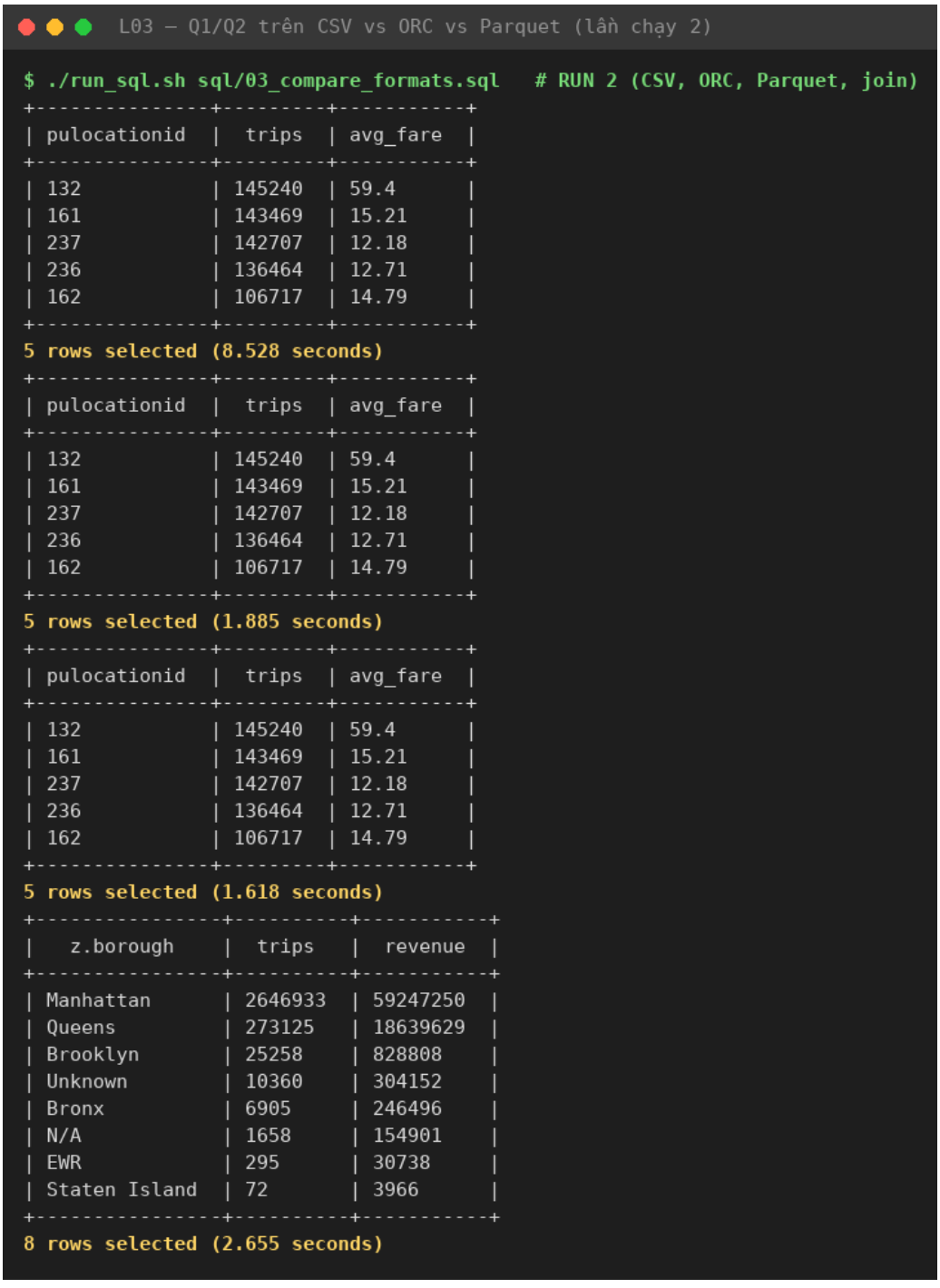
2.4 So sánh thời gian truy vấn CSV, ORC và Parquet

Q1 tìm năm zone đón khách nhiều nhất trong tháng 01/2024, chỉ dùng `pulocationid`, `fare_amount` và điều kiện thời gian. Trên CSV, điều kiện là `tpep_pickup_datetime`; trên ORC và Parquet là `pickup_month = '2024-01'`. Q2 nối `trips_orc` với `zones`. `03_compare_formats.sql` được chạy hai lần với `hive.compute.query.using.stats=false`, `hive.query.results.cache.enabled=false` và `hive.fetch.task.conversion=none` để Hive phải quét thật; số dùng để so sánh là lần thứ hai, khi JVM và Tez đã khởi động xong (Bảng 6, Hình 9).

Bảng 6. Thời gian truy vấn theo định dạng (số liệu dùng để so sánh là lần chạy thứ hai).

Truy vấn	Lần chạy	CSV	ORC	Parquet
Q1 top 5 zone đón khách	1	9.222 s	1.987 s	1.767 s
Q1 top 5 zone đón khách	2	8.528 s	1.885 s	1.618 s
Q2 nối `zones` theo borough	1		2.939 s	

Truy vấn	Lần chạy	CSV	ORC	Parquet
Q2 nối zones theo borough	2		2.655 s	



Hình 9. Lần chạy thứ hai của `03_compare_formats.sql`.

Ở lần thứ hai, ORC nhanh hơn CSV khoảng $8.528/1.885 \approx 4.5$ lần, Parquet khoảng $8.528/1.618 \approx 5.3$ lần. Mỗi truy vấn chỉ đo hai lần nên chưa nói được gì về phương sai, và với dữ liệu khoảng 274 MB thì chi phí khởi động Tez chiếm phần đáng kể trong các thời gian

1.6–2.7 s. Ba nguồn cho cùng kết quả Q1 (Bảng 7), nên đổi định dạng không làm sai dữ liệu. Zone `132` (sân bay JFK) đứng đầu, giá cước trung bình cao gấp mấy lần các zone nội đô.

Bảng 7. Kết quả Q1 (giống nhau trên CSV, ORC và Parquet).

pulocationid	Số chuyến	Giá cước trung bình
132	145,240	59.40
161	143,469	15.21
237	142,707	12.18
236	136,464	12.71
162	106,717	14.79

Theo Q2 (Bảng 8), Manhattan chiếm 89.3 % số chuyến. Cộng số chuyến các borough ra đúng 2,964,606, bằng số dòng của phân vùng `2024-01`.

Bảng 8. Kết quả Q2: số chuyến và doanh thu theo borough.

Borough	Số chuyến	Doanh thu (`total_amount`)
Manhattan	2,646,933	59,247,250
Queens	273,125	18,639,629
Brooklyn	25,258	828,808
Unknown	10,360	304,152
Bronx	6,905	246,496
N/A	1,658	154,901
EWR	295	30,738
Staten Island	72	3,966

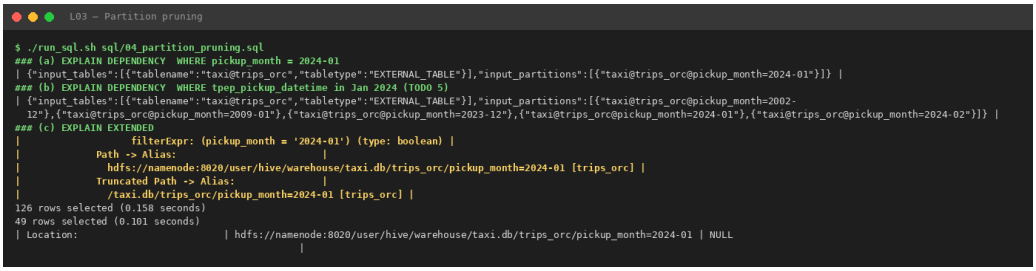
2.5 Cắt tỉa phân vùng

Ngoài hai câu `EXPLAIN` của đề, chúng tôi thêm một `EXPLAIN DEPENDENCY` lọc theo `tpep_pickup_datetime` để đối chiếu (Bảng 9, Hình 10).

Bảng 9. Kết quả kiểm chứng cắt tỉa phân vùng.

Thí nghiệm	Điều kiện lọc	Kết quả
(a) `EXPLAIN DEPENDENCY`	`pickup_month = '2024-01'`	`input_partitions` có **1** phân vùng (`pickup_month=2024-01`)
(b) `EXPLAIN DEPENDENCY`	`tpep_pickup_datetime` trong tháng 1	`input_partitions` có cả 5 phân vùng

Thí nghiệm	Điều kiện lọc	Kết quả
(c) `EXPLAIN EXTENDED`	`pickup_month = '2024-01'`	`filterExpr: (pickup_month = '2024-01')`; `Path -> Alias` chỉ có `.../trips_orc/pickup_month=2024-01`



Hình 10. Kết quả `EXPLAIN DEPENDENCY` và `EXPLAIN EXTENDED`.

Trong kế hoạch (c), `TableScan` ước lượng `Num rows: 2964606`, `Data size: 849321344`, `Basic stats: COMPLETE`, đúng bằng số dòng của riêng phân vùng `2024-01`. Cả `Map 1` và `Reducer 2` đều chạy `Execution mode: vectorized`. Ở (b), Hive không biết `tpep_pickup_datetime` và `pickup_month` có liên quan, vì trong lược đồ đó là hai cột độc lập, nên phải liệt kê mọi phân vùng rồi lọc từng dòng.

2.6 Bucketing

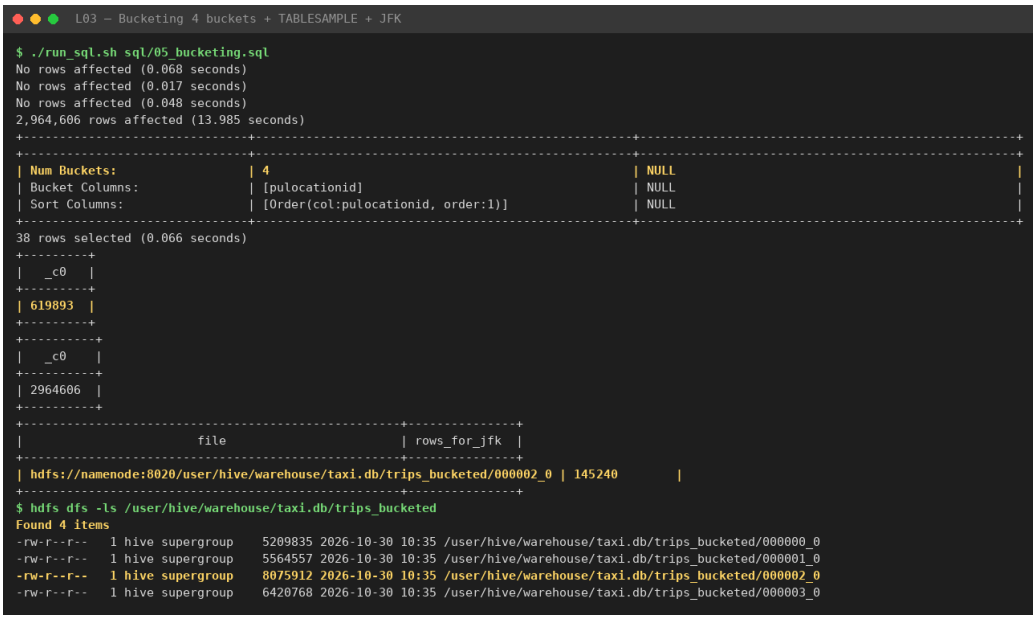
Bảng `trips_bucketed` (5 cột, `CLUSTERED BY (pulocationid) SORTED BY (pulocationid) INTO 4 BUCKETS`) nạp 2,964,606 dòng từ phân vùng `2024-01` trong 13.985 s. `DESCRIBE FORMATTED` ghi `Num Buckets: 4`, `Bucket Columns: [pulocationid]` và `Sort Columns: [Order(col:pulocationid, order:1)]`. Thư mục bảng có đúng bốn tệp (Bảng 10, Hình 12).

Bảng 10. Các tệp bucket của `trips_bucketed`.

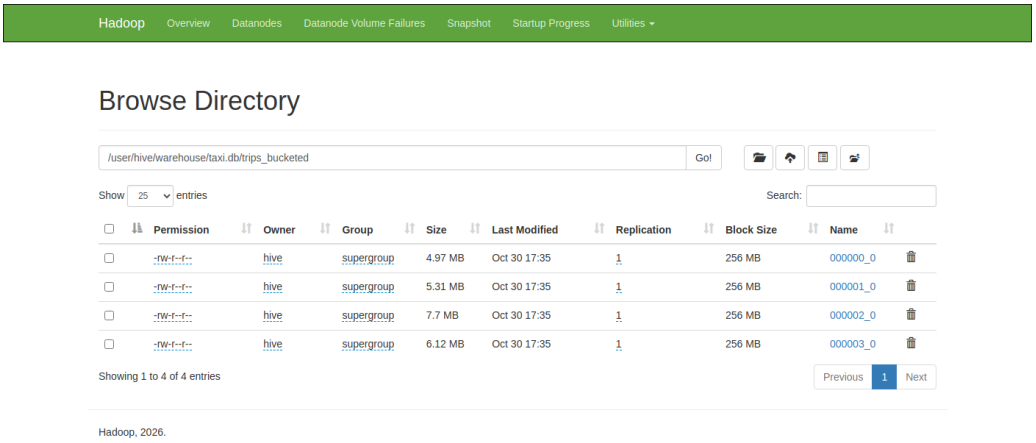
Tệp	Kích thước	Ghi chú
`000000_0`	5,209,835 B	
`000001_0`	5,564,557 B	
`000002_0`	8,075,912 B	chứa toàn bộ 145,240 dòng JFK (zone 132)
`000003_0`	6,420,768 B	
Tổng	25,271,072 B	`numFiles=4`, `numRows=2964606`

`TABLESAMPLE(BUCKET 1 OUT OF 4 ON pulocationid)` trả về 619,893 dòng sau 1.606 s, khoảng 20.9 % chứ không phải 25 %, vì hàm băm chia theo giá trị khóa chứ không theo số dòng;

bucket nào nhận các zone đông khách sẽ to hơn. Truy vấn `INPUT__FILE__NAME` cho zone 132 chỉ trả về tệp `000002_0` (Hình 11), cũng là tệp lớn nhất.



Hình 11. Kết quả bucketing: mô tả bảng, lấy mẫu một bucket và vị trí các dòng JFK.



Hình 12. Bốn tệp bucket của `trips_bucketed` trên giao diện NameNode.

2.7 Phần mở rộng: thống kê và bảng ACID

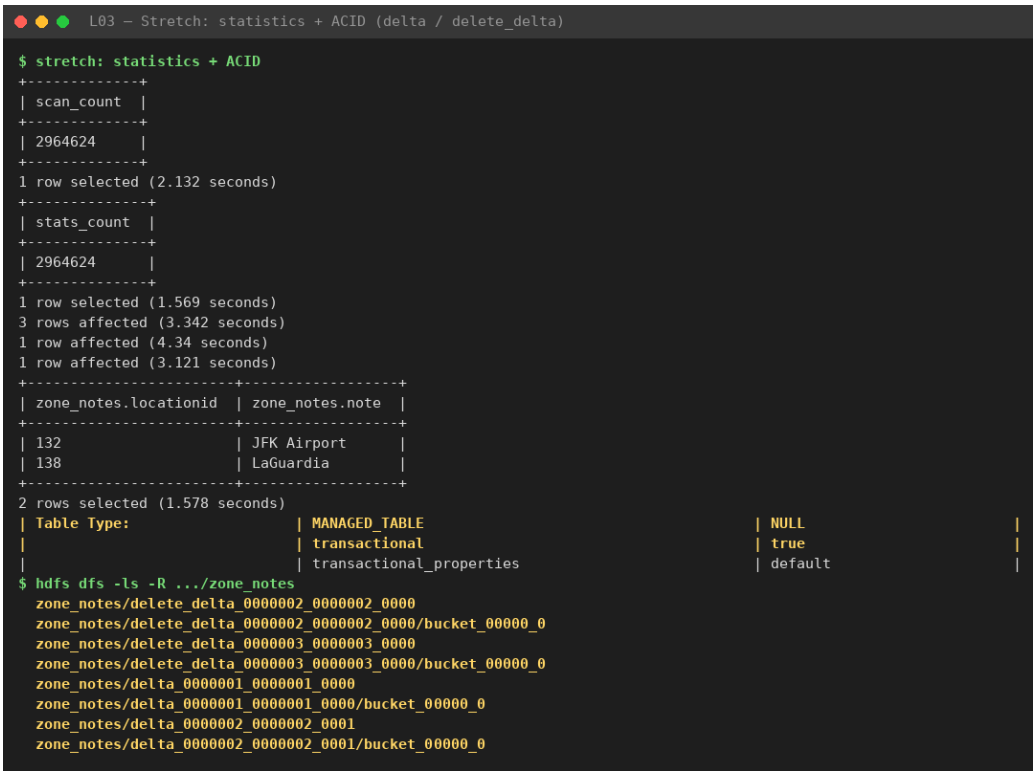
Trước khi có thống kê, `SELECT count(*)` trên `trips_orc` mất 2.132 s; câu `ANALYZE TABLE trips_orc PARTITION (pickup_month) COMPUTE STATISTICS FOR COLUMNS` mất 18.354 s. Chúng tôi chờ `count(*)` sau đó được trả lời ngay từ Metastore khi bật `hive.compute.query.`

using.stats=true, nhưng không: kết quả vẫn là 2,964,624 qua DAG Tez, mất 1.569 s lần đầu và 2.592 s khi đo lại, dù EXPLAIN có TableScan [TS_0] (rows=2964624 width=290) với Tbl:COMPLETE, Col:COMPLETE. Giải thích hợp lý nhất là trips_orc thực chất là bảng EXTERNAL được chuyển đổi, và Hive không tin thống kê của bảng ngoài vì tệp có thể bị sửa từ bên ngoài. Chúng tôi chưa kiểm chứng trên bảng ACID, nên đây vẫn là giả thuyết.

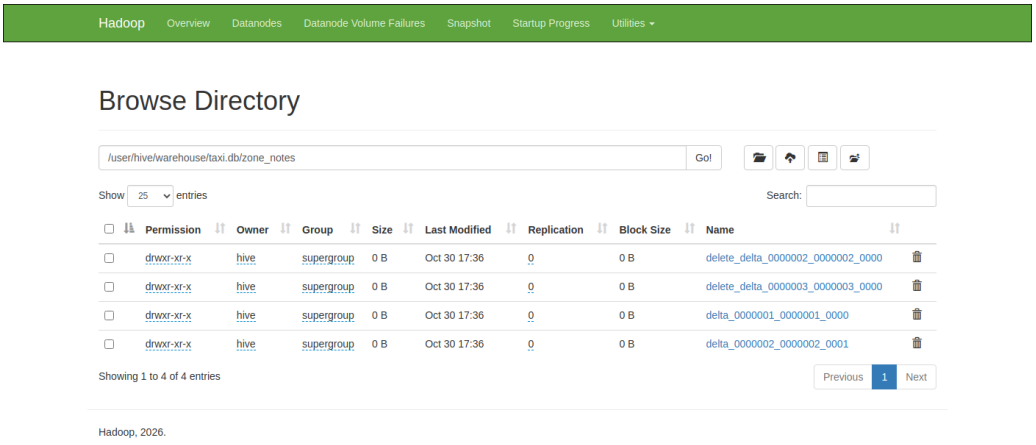
Bảng zone_notes (locationid INT, note STRING) STORED AS ORC TBLPROPERTIES ('transactional'='true') mới là MANAGED_TABLE thật. Các giao dịch và thư mục sinh ra trên HDFS nằm ở Bảng 11, Hình 13 và Hình 14. Cuối cùng còn 2 dòng: 132 | JFK Airport và 138 | LaGuardia.

Bảng 11. Các giao dịch trên bảng ACID zone_notes và thư mục delta tương ứng.

Giao dịch	Câu lệnh	Thời gian	Thư mục sinh ra
1	INSERT (132 JFK), (138 LaGuardia), (161 Midtown)	3.342 s (3 dòng)	delta_0000001_0000001_0000
2	UPDATE ... SET note='JFK Airport' WHERE locationid=132	4.34 s (1 dòng)	delete_delta_0000002_0000002_0000 + delta_0000002_0000002_0001
3	DELETE ... WHERE locationid=161	3.121 s (1 dòng)	delete_delta_0000003_0000003_0000



Hình 13. Phần mở rộng: thống kê cột và các thao tác ACID.



Hình 14. Các thư mục `delta_*` và `delete_delta_*` của bảng `zone_notes` trên giao diện NameNode.

Không tệp nào bị sửa tại chỗ: `UPDATE` là một `delete_delta` cộng một `delta` mới, mỗi thư mục có một tệp `bucket_00000_0` khoảng 700–800 B và tệp `_orc_acid_version`. Bộ đọc hợp nhất các delta khi truy vấn, compaction sẽ gộp chúng thành `base_*` về sau [4].

3 Trả lời câu hỏi checkpoint

3.1 Metastore giữ gì, HDFS giữ gì, và `DROP TABLE` xóa những gì?

Metastore (`TBLS`, `SDS`, `PARTITIONS`… trong PostgreSQL) giữ siêu dữ liệu: tên và kiểu cột, SerDe, lớp định dạng, loại bảng, `LOCATION` của bảng và từng phân vùng, thống kê (`numRows`, `totalSize`…) và trạng thái giao dịch ACID. HDFS chỉ chứa tệp dữ liệu (Bảng 3). `DROP TABLE trips_csv` chỉ xóa siêu dữ liệu, tệp trong `/data/taxi_csv` vẫn còn. `DROP TABLE trips_orc` xóa cả siêu dữ liệu lẫn thư mục `/user/hive/warehouse/taxi.db/trips_orc`, vì bảng có `external.table.purge=TRUE` (Hình 7).

3.2 Vì sao `zones` cần `OpenCSVSerde` còn `trips_csv` chỉ cần `ROW FORMAT DELIMITED`?

`ROW FORMAT DELIMITED` (`LazySimpleSerDe`) chỉ cắt trường theo dấu phân cách. Nó không bỏ dấu ngoặc kép và không hiểu dấu phẩy nằm trong ngoặc kép. Tệp trips sinh ra từ Parquet, không tiêu đề, không ngoặc kép, nên vậy là đủ. Tệp zones có dòng tiêu đề và giá trị như `"Allerton/Pelham Gardens"`, nên cần `OpenCSVSerde` cùng `skip.header.line.count=1`. Đổi lại, `OpenCSVSerde` coi mọi cột là `STRING` (Q2 phải viết `CAST(z.locationid AS INT)`), phân tích chậm hơn và không xử lý được giá trị NULL đặc biệt.

3.3 Vì sao ORC và Parquet nhỏ hơn CSV 4–5 lần?

Thứ nhất là lưu theo cột [2], [3]: giá trị cùng cột, cùng kiểu nằm cạnh nhau trong từng stripe/row group nên dictionary, *run-length encoding*, delta và bit-packing nén rất tốt, nhất là với cột ít giá trị như `vendorid`, `payment_type`. Thứ hai là kiểu nhị phân cộng nén khối: một timestamp chiếm 8 byte thay vì chuỗi 19 ký tự, rồi cả khối được nén thêm bằng ZLIB hoặc Snappy (codec là lý do chính khiến ORC và Parquet chênh nhau ở Bảng 5). Tỉ lệ `rawDataSize`/`totalSize` của `trips_orc` (Mục 2.3) cho thấy dữ liệu giải nén lớn gấp khoảng 15.7 lần trên đĩa.

3.4 Vì sao Q1 chạy nhanh hơn trên ORC và Parquet?

Bốn cơ chế cùng góp phần vào mức giảm ở Bảng 6. Column pruning: Q1 chỉ cần 2–3 trong 19 cột, ORC và Parquet chỉ đọc đúng các column chunk đó, còn CSV phải phân tích cả 274 MB từng dòng. Partition pruning: điều kiện trên `pickup_month` khiến Hive chỉ mở thư mục `pickup_month=2024-01` (Bảng 9), còn bảng CSV không phân vùng nên phải quét hết. Predicate pushdown: min/max của từng stripe/row group cho phép bỏ qua khối không thỏa điều kiện. Cuối cùng là thực thi vectorized theo lô 1,024 dòng (Mục 2.5), không phải đổi văn bản thành số. Thí nghiệm không tách riêng được đóng góp của từng cơ chế.

3.5 Phân vùng tĩnh khác phân vùng động ra sao, và vì sao mặc định là `strict`?

Phân vùng tĩnh ghi giá trị ngay trong câu lệnh, ví dụ `PARTITION (pickup_month='2024-01')`, và mọi dòng đều vào phân vùng đó. Phân vùng động lấy giá trị từ cột cuối của `SELECT`. Chế độ `strict` bắt phải có ít nhất một giá trị tĩnh, để một câu truy vấn viết sai không lỡ tay tạo ra hàng nghìn phân vùng, hàng nghìn tệp và đối tượng trong Metastore. Bài này gặp đúng chuyện đó ở quy mô nhỏ: 18 dòng bất thường đã sinh ra bốn phân vùng lạ (Bảng 4).

3.6 Phân vùng theo `pulocationid` cộng ngày, hoặc theo `tpep_pickup_datetime`, thì sao?

Theo `pulocationid` cộng ngày sẽ có khoảng $265 \times 31 \approx 8{,}000$ phân vùng mỗi tháng, mỗi phân vùng một tệp từ vài KB đến vài trăm KB, tức vấn đề tệp nhỏ (*small files problem*) [5]. NameNode phải giữ hàng chục nghìn đối tượng trong heap, Metastore phải lưu hàng nghìn dòng `PARTITIONS`/`SDS` nên lập kế hoạch chậm đi, và mỗi tệp nhỏ thành một task riêng nên chi phí khởi tạo lấn át xử lý thật. Bốn phân vùng lẻ khoảng 2 KB của `trips_orc` (Bảng 4) là dấu hiệu sớm. Phân vùng theo timestamp gốc còn tệ hơn: gần như mỗi giây một phân vùng, hàng triệu thư mục chứa tệp vài trăm byte. Nên chọn cột ít giá trị mà truy vấn hay lọc, sao cho mỗi phân vùng cỡ vài trăm MB đến khoảng 1 GB.

3.7 Khi nào dùng partitioning, khi nào dùng bucketing, và vì sao số tệp bucket cố định?

Partitioning tạo thư mục theo giá trị cột, hợp với cột ít giá trị mà truy vấn hay lọc (ngày, tháng, quốc gia), để bỏ qua nguyên thư mục khi đọc. Bucketing chia theo hash(c) mod N vào N tệp, hợp với cột nhiều giá trị như mã định danh hay `pulocationid`; nó cho phép lấy mẫu bằng `TABLESAMPLE`, và bucket map join hay sort-merge-bucket join không cần shuffle khi hai bảng cùng bucket trên khóa join. Số tệp cố định vì N đã chốt trong DDL (`INTO 4 BUCKETS`): mỗi dòng luôn rơi vào bucket hash mod 4 dù dữ liệu nhiều hay ít, đúng như Bảng 10. Hai kỹ thuật dùng chung được, ví dụ phân vùng theo tháng rồi chia bucket theo zone.

3.8 Những dòng nào rơi vào phân vùng lạ, và pipeline thật nên xử lý ra sao?

18 dòng ngoài tháng 01/2024 (Hình 7) được tóm lại ở Bảng 12.

Bảng 12. Các dòng có thời gian đón khách nằm ngoài tháng 01/2024.

Phân vùng	Số dòng	Mô tả	Đánh giá
2002-12	2	2002-12-31 22:59:39, zone 170, một cặp `total_amount` +10.5/−10.5	Lỗi đồng hồ; một chuyến bị hoàn tiền
2009-01	3	Ngày 2009-01-01, cả ba trả khách tại zone 264	Lỗi đồng hồ thiết bị
2023-12	10	2023-12-31 23:39–23:58	Chuyến thật bắt đầu trước giao thừa
2024-02	3	2024-02-01 00:00:17–00:01:15	Chuyến thật bắt đầu ngay sau nửa đêm

Dòng năm 2002 và 2009 là lỗi đồng hồ, còn dòng 2023-12 và 2024-02 là chuyến thật ở biên tháng. Trong pipeline thật, chúng tôi sẽ so thời gian sự kiện với tháng của tệp nguồn, đẩy dòng ngoài khoảng hợp lệ vào một bảng quarantine kèm lý do và cảnh báo khi số dòng này tăng; dòng biên tháng hợp lệ thì ghi vào đúng phân vùng của nó hoặc giữ theo tháng của tệp nguồn, tùy quy ước. Không nên để phân vùng động lặng lẽ tạo phân vùng rác.

4 Kết luận

Bài đã hoàn thành đủ các task của đề cùng hai phần mở rộng (thống kê, ACID). Định dạng cột giảm kích thước 4–5 lần và thời gian Q1 tương ứng so với CSV (Bảng 5, Bảng 6); Hive chỉ cắt tỉa phân vùng khi điều kiện lọc nằm đúng trên cột phân vùng (Bảng 9). Hai điểm lệch so với kỳ vọng là phân vùng thứ năm `2024-02` và việc `count(*)` không dùng thống kê. Mọi thứ chạy trên một máy với Tez cục bộ và một DataNode, nên các con số không phản ánh tính song song hay chi phí mạng của cụm thật.

Tài liệu tham khảo

[1] Apache Software Foundation, “Apache Hive Language Manual: DDL,” Apache Hive Wiki. https://cwiki.apache.org/confluence/display/Hive/LanguageManual+DDL

[2] Apache Software Foundation, “ORC Specification v1,” Apache ORC. https://orc.apache.org/specificatio

[3] Apache Software Foundation, “Apache Parquet Documentation,” Apache Parquet. https://parquet.apache.org/docs/

[4] Y. Huai, A. Chauhan, A. Gates, G. Hagleitner, E. N. Hanson, O. O'Malley, J. Pandey, Y. Yuan, R. Lee, X. Zhang, "Major Technical Advancements in Apache Hive," in *Proc. ACM SIGMOD*, 2014, pp. 1235–1246.

[5] T. White, *Hadoop: The Definitive Guide*, 4th ed. Sebastopol, CA: O'Reilly Media, 2015.

Phụ lục: Trích đoạn mã nguồn

Các tệp nộp kèm trong `submission/L03/` được tóm ở Bảng 13.

Bảng 13. Danh mục tệp nộp kèm trong `submission/L03/`.

Tệp	Nội dung
`01_external_tables.sql` … `05_bucketing.sql`	Các tệp SQL đã hoàn thành
`01_stack_and_upload.txt` … `10_bucketing.txt`	Log từng bước, theo thứ tự thực hiện
`11_stretch_stats_acid.txt`, `12_stretch_stats_answer.txt`	Phần mở rộng thống kê và ACID
`img/`	Ảnh chụp terminal và giao diện web

Trong `01_external_tables.sql`, bảng `zones` dùng `OpenCSVSerde` và bỏ dòng tiêu đề:

```
CREATE EXTERNAL TABLE zones (
  locationid STRING, borough STRING, zone STRING, service_zone STRING
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
STORED AS TEXTFILE
LOCATION '/data/zones'
TBLPROPERTIES ('skip.header.line.count' = '1');
```

Trong `02_partitioned_tables.sql`, bật phân vùng động, tính `pickup_month` ở cột cuối của `SELECT` và khai báo bảng Parquet nén Snappy (câu nạp `trips_parquet` giống hệt câu nạp `trips_orc`):

```
SET hive.exec.dynamic.partition.mode = nonstrict;

INSERT OVERWRITE TABLE trips_orc PARTITION (pickup_month)
SELECT t.*,
       date_format(t.tpep_pickup_datetime, 'yyyy-MM') AS pickup_month
FROM trips_csv t;
```

```
CREATE TABLE trips_parquet ( ... cùng 19 cột như trips_orc ... )
PARTITIONED BY (pickup_month STRING)
STORED AS PARQUET
TBLPROPERTIES ('parquet.compression' = 'SNAPPY');
```

Trong `05_bucketing.sql`, bảng bucketing chia theo `pulocationid` vào 4 bucket:

```
CREATE TABLE trips_bucketed (
  tpep_pickup_datetime TIMESTAMP, pulocationid INT, dolocationid INT,
  fare_amount DOUBLE, total_amount DOUBLE
)
CLUSTERED BY (pulocationid) SORTED BY (pulocationid) INTO 4 BUCKETS
STORED AS ORC;
```